

EduGenie – AI-Powered Multimodal Educational Assistant

1 System Overview

EduGenie is a Flask-based web application that integrates the Google Gemini Generative AI API to provide:

- AI-based question answering
- Quiz generation
- Multilingual responses

2 Problem Domain

Students and learners need:

- Automatic quiz generation
- Manual preparation of quizzes and summaries is time-consuming.

3 Core Technology Used

Backend Framework

- Flask (Python Web Framework)

Programming Language

- Python 3

AI Service

- Google Gemini API (via google-generativeai library)

4 Key Capabilities

- AI Question Answering
- Quiz generation in structured JSON
- Multilingual support

5 Target Users

- School students
- College students
- Teachers

6 Real-World Relevance

- AI learning assistants
- Smart tutoring systems

3. APPLICATION SCENARIOS / USE CASES

Student Usage

- Enter topic → Generate MCQs

4 TECHNICAL ARCHITECTURE OVERVIEW

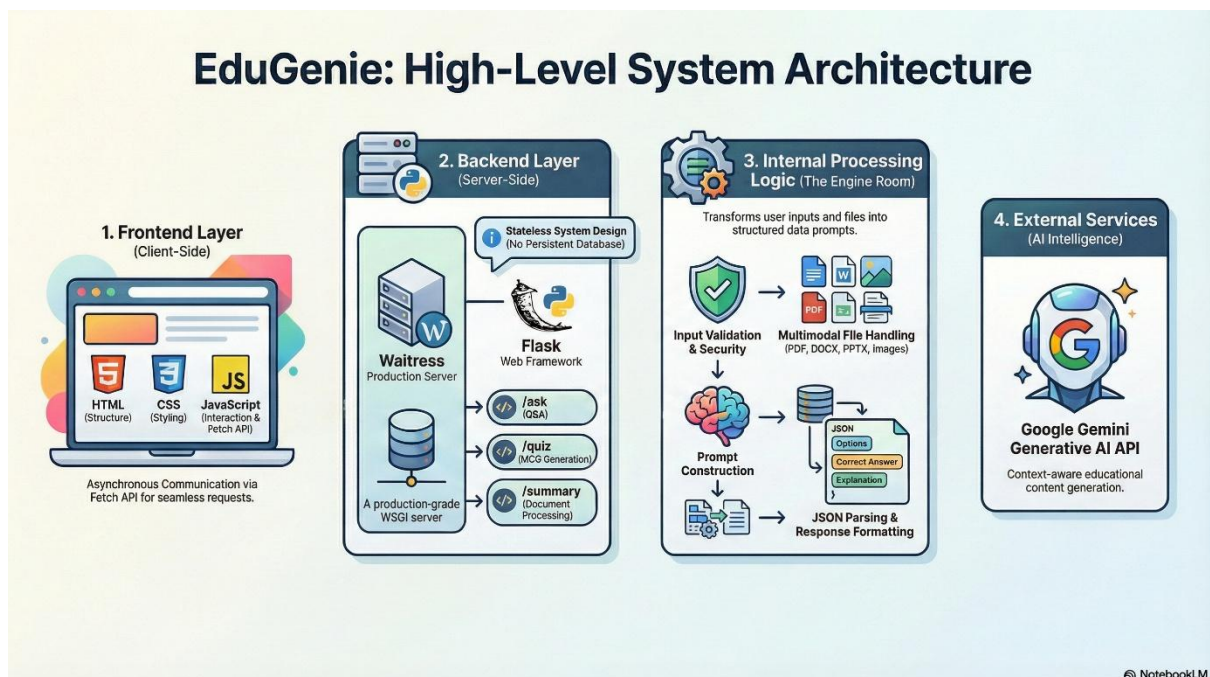


Fig.1 High level architecture diagram

Inside EduGenie: The Stateless Architecture of an AI Learning Assistant

Mapping the technical component interactions and data flow of the EduGenie system, where user inputs are transformed into AI-driven educational outputs without persistent data storage.

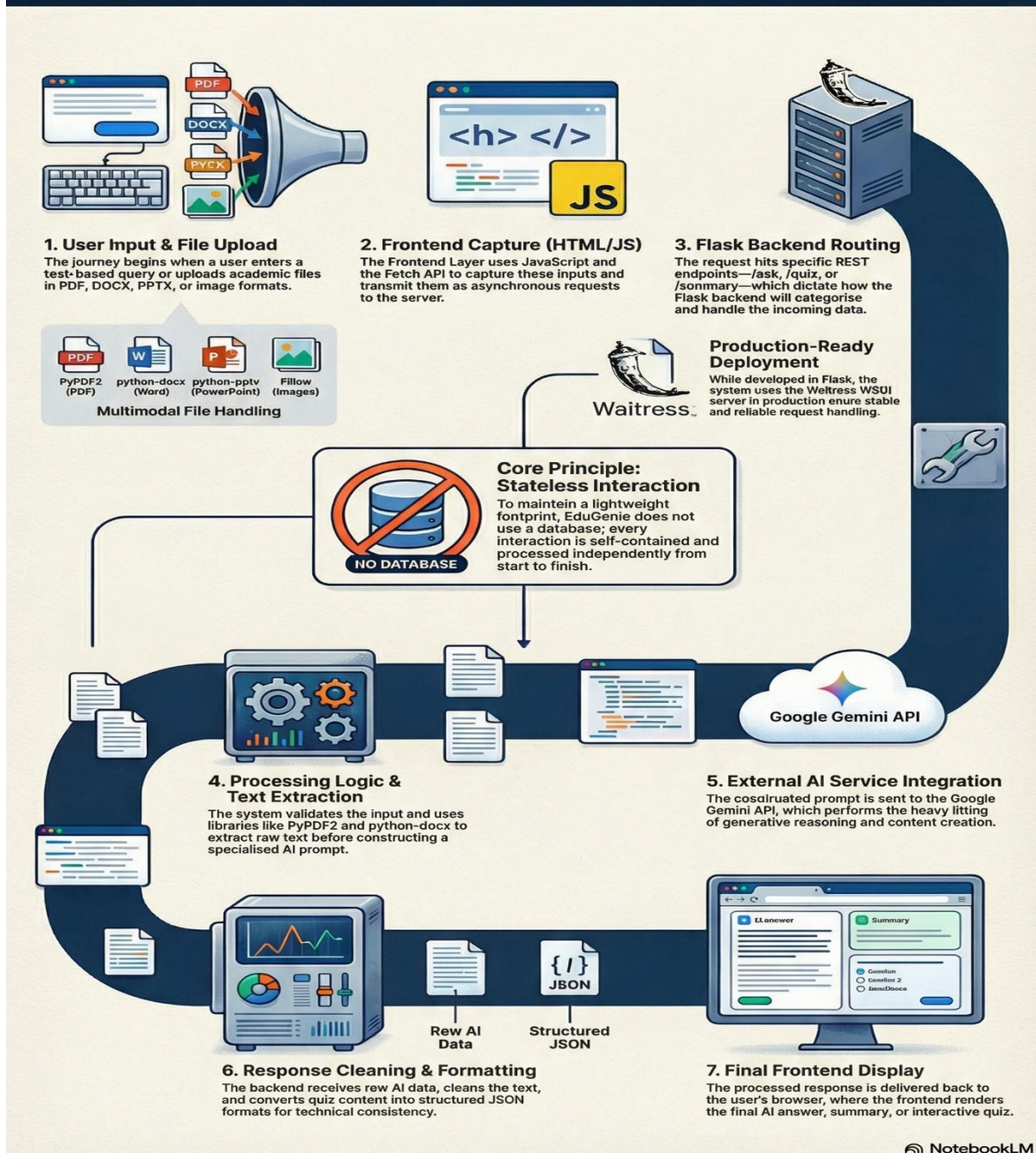


Fig 2 Component Interaction diagram

1 System Overview

EduGenie is a Flask-based web application that integrates the Google Gemini Generative AI API to provide:

- AI-based question answering
- Quiz generation
- Multilingual responses

2 Problem Domain

Students and learners need:

- Automatic quiz generation
- Manual preparation of quizzes and summaries is time-consuming.

3 Core Technology Used

Backend Framework

- Flask (Python Web Framework)

Programming Language

- Python 3

AI Service

- Google Gemini API (via google-generativeai library)

4 Key Capabilities

- AI Question Answering
- Quiz generation in structured JSON
- Multilingual support

5 Target Users

- School students
- College students
- Teachers

6 Real-World Relevance

- AI learning assistants

- Smart tutoring systems

3. APPLICATION SCENARIOS / USE CASES

Student Usage

- Enter topic → Generate MCQs

4 TECHNICAL ARCHITECTURE OVERVIEW

4.1 Frontend Layer

- HTML templates
- JavaScript (for form submission & fetch API)
- Basic CSS styling

4.2 Backend Layer

- Built using:
- Flask
- REST API routes:
- /ask
- /quiz
- /summary
- /models
- Responsibilities:
- Input validation
- File handling
- Prompt construction
- Gemini API call
- JSON parsing
- Error handling

4.3 External APIs / Services

- Google Gemini Generative AI API

4.4 Deployment Environment

- Local development (Flask)
- Production server: Waitress

5. PREREQUISITES

5.1 Software Requirements

- Python 3.10+
- pip
- Virtual environment (venv)
- Web browser
- Gemini API Key
- .env file

5.2 Libraries / Frameworks / Dependencies

- flask
- google-generativeai
- python-dotenv

5.3 Hardware Requirements

- Minimum: 4GB RAM
- Internet connection

6. PRIOR KNOWLEDGE REQUIRED

- Python programming
- Flask basics
- REST APIs
- JSON handling
- Environment variables
- Basic understanding of Generative AI

7. PROJECT OBJECTIVES

7.1 Technical Objectives

- Integrate Gemini API with Flask
- Handle multimodal input
- Generate structured quiz output
- Provide multilingual AI responses

7.2 Performance Objectives

- Quick response generation
- Efficient text extraction
- Proper error handling

7.3 Deployment Objectives

- Run locally
- Production-ready with Waitress

7.4 Learning Outcomes

- API integration
- AI application development
- Flask web development
- Prompt engineering
- File processing in Python

8. SYSTEM WORKFLOW

Step 1: User Interaction - User enters text or uploads file.

Step 2: Input Handling - Flask receives request and extracts text if file is uploaded.

Step 3: Processing Logic - Prompt is constructed and sent to Gemini API.

Step 4: Core System Execution - Gemini model generates response.

Step 5: Output Generation - Response is cleaned and formatted (JSON if quiz).

Step 6: Response Delivery - Backend sends JSON response → Frontend displays result.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

In the current education system, students and teachers frequently deal with large volumes of study material in formats such as PDFs, Word documents, PowerPoint presentations, and images. Extracting key concepts, generating summaries, and preparing assessment questions manually is time-consuming and inefficient. There is a need for an intelligent system that can automatically process academic content and generate meaningful explanations, summaries, and quizzes. EduGenie addresses this problem by integrating a Flask-based backend with the Gemini Generative AI API provided by Google to automate educational content understanding and quiz generation through a simple web interface.

9.2 Functional Requirements

The system must allow users to input academic questions and receive AI-generated responses. It must support file uploads in PDF, DOCX, PPTX, and image formats and extract readable text from these files. The application must provide quiz generation functionality that returns structured multiple-choice questions in JSON format. The backend should expose REST endpoints such as `/ask`, `/quiz`, `/summary`, and `/models` to handle different request types. Additionally, the system must validate user input, handle errors gracefully, and return structured responses to the frontend interface.

9.3 Non-Functional Requirements

The system should deliver responses within acceptable time limits depending on external API latency. It must ensure secure handling of API keys using environment variables managed through `python-dotenv`. The application should run on a standard machine with at least 4GB RAM and an active internet connection. The system must be reliable, maintainable, and easy to deploy using a production WSGI server such as Waitress. Since no database is used, the system must operate in a stateless manner and process each request independently without storing user data.

9.4 System Design Decisions

EduGenie follows a client-server architecture. A lightweight Flask framework was chosen to implement RESTful API endpoints efficiently. The decision to avoid a database was made to keep the application simple and focused on AI-driven processing rather than data storage. File-processing libraries such as `PyPDF2`, `python-docx`, `python-pptx`, and `Pillow` were selected to support commonly used educational document formats. The backend is

responsible for input validation, text extraction, prompt construction, API communication, and response formatting, while the frontend handles user interaction and display. Waitress was selected as the production server instead of Flask's development server to ensure stable deployment.

9.5 Technology Stack Selection Justification

Python was selected due to its extensive ecosystem for AI integration and file processing. Flask was chosen because of its simplicity, flexibility, and suitability for REST API development. The Gemini Generative AI service from Google was selected to provide advanced text generation, summarisation, and quiz creation capabilities. The Google-generative AI library simplifies communication with the AI service. PyPDF2, python-docx, and python-pptx were chosen to extract text from academic documents, while Pillow supports image handling. The python-dotenv library ensures secure configuration management of API keys. Waitress was selected as a production-ready WSGI server to enhance deployment reliability.

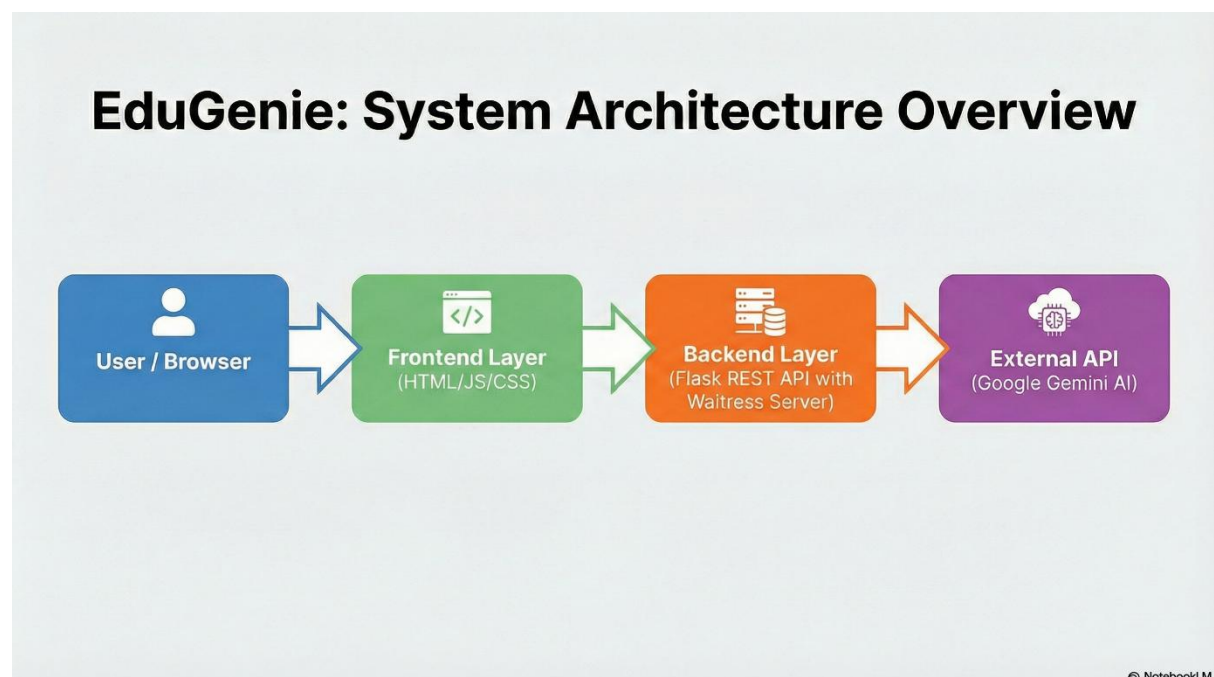


Fig 3. Architecture Diagram

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

The EduGenie system was developed using Python 3.10 and the Flask web framework. A virtual environment was created to manage dependencies separately from the system Python installation. The development was carried out using Visual Studio Code as the Integrated Development Environment (IDE). An active internet connection is required to access the Gemini Generative AI API provided by Google.

Setup Commands

Open Command Prompt or Terminal and execute:

```
python --version
```

```
python -m venv venv
```

```
venv\Scripts\activate (Windows)
```

10.2 Dependency Installation

After activating the virtual environment, the required dependencies were installed using pip.

The following libraries are used in the project:

- Flask
- google-generative AI
- python-dotenv
- PyPDF2
- python-docx
- python-pptx
- Pillow
- waitress

Installation Command: `pip install -r requirements.txt`

10.3 Project Structure Creation

The project follows the below directory structure:

EduGenie/

```
|
|— app.py
|— requirements.txt
|— .env
|— templates/
|   |— index.html
|— static/
|   |— (CSS / JS files)
|— venv/
```

10.4 Configuration Setup

The Gemini API key is stored securely inside a .env file.

Create a .env file in the project root:

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

This milestone focuses on the implementation of the core functionalities of the EduGenie system. The main features include AI-based question answering, quiz generation, and document summarization. These features are implemented using a Flask backend integrated with the Gemini Generative AI API provided by Google.

11.1 Feature 1 – AI-Based Question Answering

Description:

This feature allows users to enter academic questions through the web interface and receive AI-generated answers. The system constructs a prompt based on the user input and sends it to the Gemini API, which returns a context-aware response. This feature helps students clarify doubts instantly without manual searching.

Code Explanation:

When a user submits a question, the Flask backend receives the request through the /ask endpoint. The input is validated to ensure it is not empty. A prompt is then constructed and sent to the Gemini API using the Google Generative AI library. The AI-generated response is processed and returned to the frontend in JSON format

Screenshots / Outputs

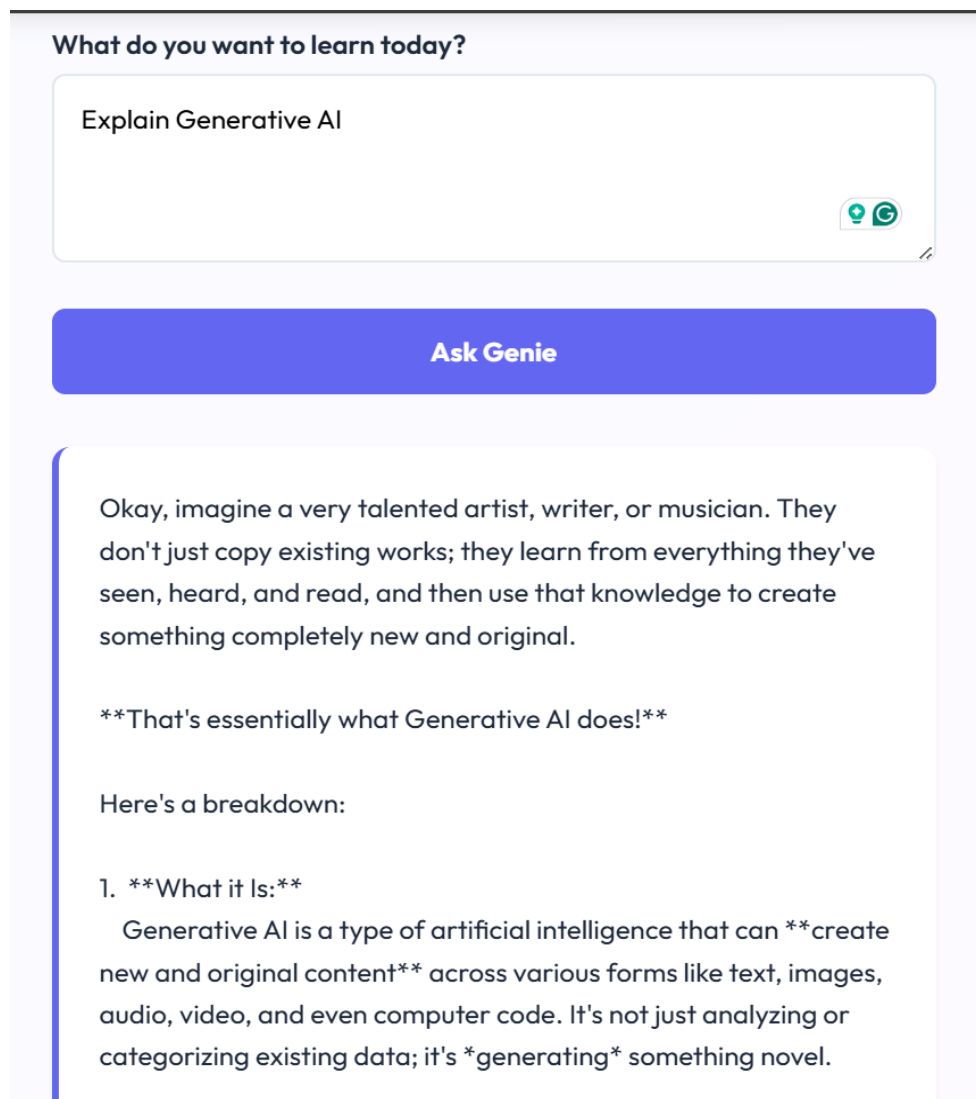


Fig 4 Question Answering Output

11.2 Feature 2 – Quiz Generation

Description:

This feature enables automatic generation of multiple-choice questions (MCQs) from a given topic or uploaded document content. It is designed to support self-assessment and practice for students.

Code Explanation:

The quiz generation logic is implemented through the /quiz endpoint. The backend accepts a topic or extracted text from uploaded files. A structured prompt is created requesting the AI to generate MCQs in JSON format. The response received from the Gemini API is parsed and validated before being sent back to the frontend. Each quiz includes questions, options, correct answers, and explanations.

Screenshots / Output

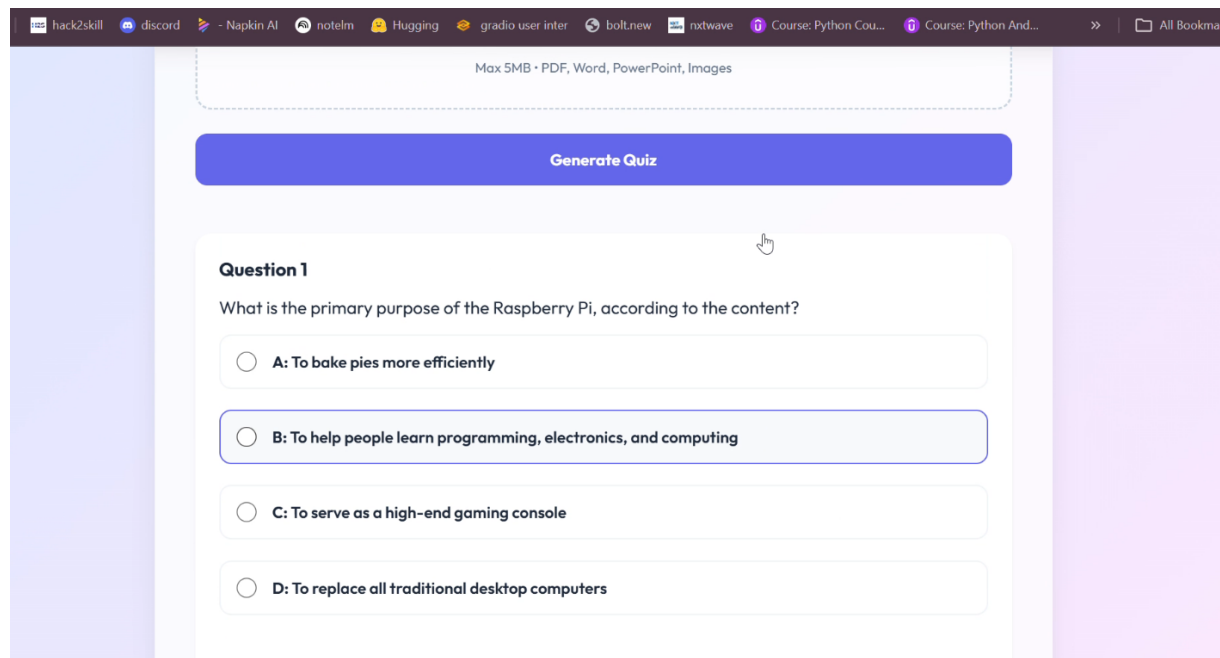


Fig 5 Quiz output

11.3 Feature 3 – Document Summarization

Description:

This feature allows users to upload documents such as PDFs, DOCX files, PPTX files, or images and receive concise summaries. It helps users quickly understand lengthy study materials.

Code Explanation:

The summarization feature is implemented using the /summary endpoint. Uploaded files are processed using appropriate libraries such as PyPDF2, python-docx, python-pptx, or Pillow to extract text. The extracted content is sent to the Gemini API with a summarization prompt. The generated summary is returned to the user as a text response.

Screenshots / Outputs:

The screenshot displays the EduGenie web interface. At the top, there are three buttons: "Ask Anything", "Generate Quiz", and "Summarize Content". Below these, there are two tabs: "Text Content" and "Upload File". Under the "Text Content" tab, there is a text input area with the prompt "Paste your study material here". The input area contains a list of four items:

- 17. The system workflow moves from input handling to prompt construction and API execution.
- 18. Performance is optimized through input validation and efficient prompt engineering.
- 19. EduGenie serves as a smart tutoring system, enhancing modern educational workflows.
- 20. It provides a practical, scalable foundation for future AI-driven EdTech solutions.

Below the input area is a blue button labeled "Summarize Material". At the bottom, there is a text box containing the summarized output:

EduGenie is an AI-powered, multimodal educational assistant built with the Flask web framework and integrating the Google Gemini Generative AI API.

Fig 6 Text Summarization output

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

All system components were integrated using a client–server architecture. The frontend sends user requests to the Flask backend, which processes input, extracts text from uploaded files, and communicates with the Gemini Generative AI API provided by Google. The AI response is formatted and returned to the user interface.

12.2 Performance Optimization

Performance was improved by validating inputs before API calls, minimizing unnecessary data processing, structuring prompts efficiently, and deploying the application using Waitress for stable request handling. The system remains lightweight and stateless for better speed and efficiency.

12.3 Security Enhancements

The API key is stored securely in a .env file and loaded using python-dotenv. Input and file validations are implemented to prevent invalid requests. No user data is stored, reducing security risks.

12.4 Error Handling & Validation

Error handling is implemented using validation checks and try–except blocks. The system manages invalid inputs, unsupported file formats, API failures, and JSON parsing errors gracefully without crashing.

13. MILESTONE 5: TESTING & VALIDATION

Test Case ID Input		Expected Output	Status
TC01	Valid question in /ask	AI-generated answer displayed	Passed
TC02	Topic in /quiz	Structured MCQ JSON response	Passed
TC03	Upload PDF file in /summary	Generated summary text	Passed
TC04	Empty input	Validation error message	Passed
TC05	Unsupported file format	File type error message	Passed

13.2 Unit Testing

Unit testing was performed on individual modules such as input validation, file extraction (PDF, DOCX, PPTX), API communication, and JSON parsing. Each function was tested independently to ensure correct output for valid and invalid inputs.

13.3 Integration Testing

Integration testing was conducted to verify that frontend forms, backend routes, file processing modules, and the Gemini Generative AI API provided by Google work together seamlessly. All endpoints (/ask, /quiz, /summary) were tested for proper request–response flow.

13.4 User Acceptance Testing (UAT)

The system was tested by students to ensure ease of use, accuracy of responses, and proper display of outputs. Feedback confirmed that the application is user-friendly and provides meaningful academic assistance.

13.5 Performance Metrics

- Average response time depends on API latency (internet-based).
- System runs smoothly on 4GB RAM.
- No database overhead, ensuring lightweight performance.
- Stable execution using Waitress in production mode.

14. DEPLOYMENT

14.1 Deployment Architecture

EduGenie follows a simple web deployment architecture. The client (user browser) sends HTTP requests to the deployed Flask backend server. The backend processes the request, communicates with the Gemini Generative AI API provided by Google, and returns the AI-generated response to the user. The system operates as a stateless web application without a database.

14.2 Hosting Platform

The application can be deployed on cloud platforms such as:

- Render

- Railway
- AWS EC2
- Google Cloud
- Heroku

14.3 Deployment Steps

1. Install Python and project dependencies.
2. Set up the .env file with the Gemini API key.
3. Activate the virtual environment.
4. Install required packages using `pip install -r requirements.txt`.
5. Run the application using Waitress:
6. `waitress-serve --host=0.0.0.0 --port=5000 app:app`
7. Configure hosting platform settings (port, environment variables).

14.4 Production Considerations

- Secure storage of API keys using environment variables.
- Use of Waitress instead of Flask development server.
- Proper input validation and file-type restrictions.
- Stable internet connection for API communication.
- Scalability depends on hosting platform resources.

16. RESULTS

● System Output

The EduGenie system successfully generates AI-based answers, structured MCQ quizzes, and concise summaries from text and uploaded documents. All endpoints (/ask, /quiz, /summary) function correctly and return meaningful responses using the Gemini Generative AI API provided by Google.

● Performance Evaluation

The system performs efficiently on a standard system (4GB RAM). Response time mainly depends on internet speed and external API latency. Since no database is used, internal processing is lightweight and fast.

- Screenshots

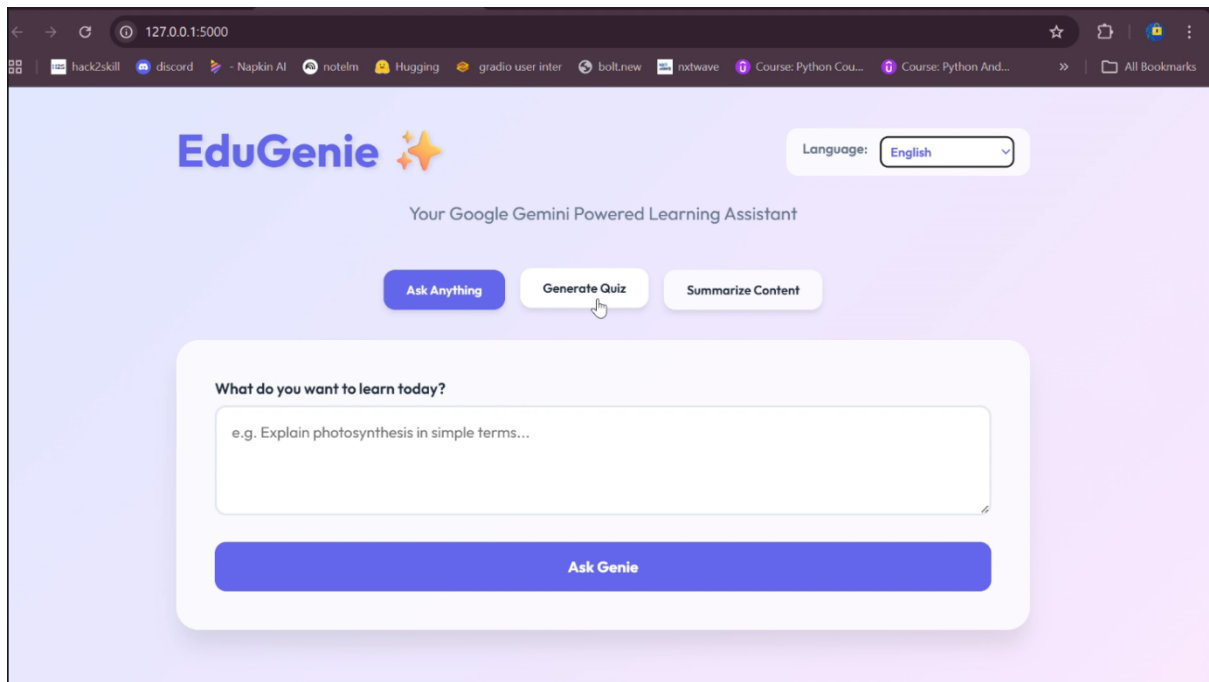


Fig 7. Home Page

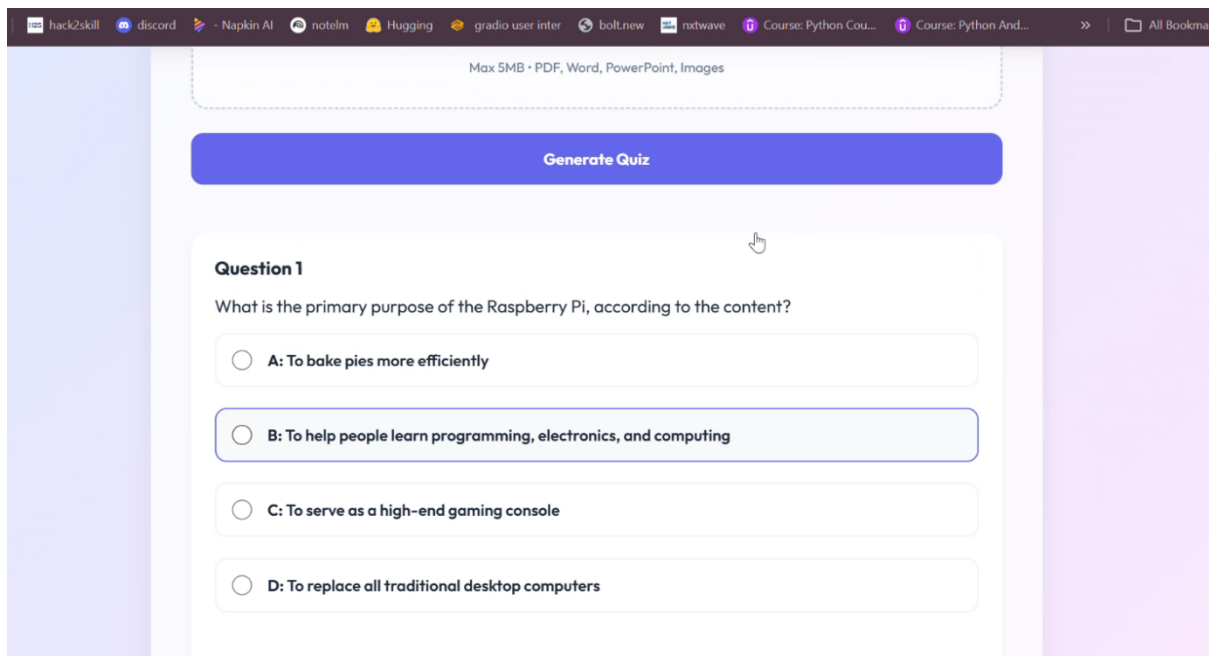


Fig 8. Generated quiz

- **Benchmark Results**

- Average response time: Few seconds (API dependent)
- Low memory usage (stateless system)
- Stable performance during testing

17. ADVANTAGES & LIMITATIONS

Advantages

- Provides instant AI-based academic assistance.
- Supports question answering, quiz generation, and document summarization.
- Accepts multiple file formats (PDF, DOCX, PPTX, images).
- Lightweight and stateless (no database required).
- Secure API key management using environment variables.
- Easy deployment using Flask and Waitress.
- User-friendly web interface.

Limitations

- Requires stable internet connection (API dependent).
- Response time depends on external API latency.
- No user login or data storage functionality.
- Accuracy depends on AI-generated content.
- Limited scalability without advanced cloud infrastructure.

18. FUTURE ENHANCEMENTS

Scalability Improvements

The system can be enhanced by deploying it on scalable cloud infrastructure with load balancing and auto-scaling features. Adding a database for storing user history and quizzes would improve long-term usability.

Feature Expansion

Future versions can include user authentication, personalized learning dashboards, chat-based interaction, multilingual UI support, and advanced analytics for tracking learning progress.

Cloud Integration

EduGenie can be fully deployed on cloud platforms such as AWS or Google Cloud for better availability, scalability, and secure environment management. Containerization using Docker can also improve portability.

Mobile Integration

A dedicated Android or iOS application can be developed to provide mobile access. Alternatively, the web application can be converted into a Progressive Web App (PWA) for mobile-friendly usage.

Automation

Automation can be implemented for scheduled quiz generation, automatic content updates, usage analytics, and integration with Learning Management Systems (LMS) for seamless academic workflows.

19. CONCLUSION

● Summary of Implementation

EduGenie was successfully developed as a Flask-based web application integrating the Gemini Generative AI API provided by Google. The system implements AI-based question answering, quiz generation, and document summarization with support for multiple file formats. All core modules were integrated, tested, and deployed using a production-ready setup.

● Technical Achievements

The project demonstrates successful integration of a web framework with a generative AI service. It includes secure API key management, structured JSON output generation,

multimodal input handling, and deployment using Waitress. The system operates efficiently without a database, maintaining a lightweight and stateless architecture.

- Practical Impact

EduGenie provides instant academic assistance to students and educators, reducing manual effort in preparing notes and quizzes. It showcases the practical application of generative AI in education and serves as a scalable foundation for future EdTech solutions.

4.1 Frontend Layer

- HTML templates
- JavaScript (for form submission & fetch API)
- Basic CSS styling

4.2 Backend Layer

- Built using:
- Flask
- REST API routes:
- /ask
- /quiz
- /summary
- /models
- Responsibilities:
- Input validation
- File handling
- Prompt construction
- Gemini API call
- JSON parsing
- Error handling

4.3 External APIs / Services

- Google Gemini Generative AI API

4.4 Deployment Environment

- Local development (Flask)
- Production server: Waitress

5. PREREQUISITES

5.1 Software Requirements

- Python 3.10+
- pip
- Virtual environment (venv)
- Web browser
- Gemini API Key
- .env file

5.2 Libraries / Frameworks / Dependencies

- flask
- google-generativeai
- python-dotenv

5.4 Hardware Requirements

- Minimum: 4GB RAM
- Internet connection

6. PRIOR KNOWLEDGE REQUIRED

- Python programming
- Flask basics
- REST APIs
- JSON handling
- Environment variables
- Basic understanding of Generative AI

7. PROJECT OBJECTIVES

7.2 Technical Objectives

- Integrate Gemini API with Flask
- Handle multimodal input
- Generate structured quiz output
- Provide multilingual AI responses

7.2 Performance Objectives

- Quick response generation
- Efficient text extraction
- Proper error handling

7.3 Deployment Objectives

- Run locally
- Production-ready with Waitress

7.4 Learning Outcomes

- API integration
- AI application development
- Flask web development
- Prompt engineering
- File processing in Python

8. SYSTEM WORKFLOW

Step 1: User Interaction - User enters text or uploads file.

Step 2: Input Handling - Flask receives request and extracts text if file is uploaded.

Step 3: Processing Logic - Prompt is constructed and sent to Gemini API.

Step 4: Core System Execution - Gemini model generates response.

Step 5: Output Generation - Response is cleaned and formatted (JSON if quiz).

Step 6: Response Delivery - Backend sends JSON response → Frontend displays result.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

In the current education system, students and teachers frequently deal with large volumes of study material in formats such as PDFs, Word documents, PowerPoint presentations, and images. Extracting key concepts, generating summaries, and preparing assessment questions manually is time-consuming and inefficient. There is a need for an intelligent system that can automatically process academic content and generate meaningful explanations, summaries, and quizzes. EduGenie addresses this problem by integrating a Flask-based backend with the Gemini Generative AI API provided by Google to automate educational content understanding and quiz generation through a simple web interface.

9.2 Functional Requirements

The system must allow users to input academic questions and receive AI-generated responses. It must support file uploads in PDF, DOCX, PPTX, and image formats and extract readable text from these files. The application must provide quiz generation functionality that returns structured multiple-choice questions in JSON format. The backend should expose REST endpoints such as /ask, /quiz, /summary, and /models to handle different request types. Additionally, the system must validate user input, handle errors gracefully, and return structured responses to the frontend interface.

9.3 Non-Functional Requirements

The system should deliver responses within acceptable time limits depending on external API latency. It must ensure secure handling of API keys using environment variables managed through python-dotenv. The application should run on a standard machine with at least 4GB RAM and an active internet connection. The system must be reliable, maintainable, and easy to deploy using a production WSGI server such as Waitress. Since no database is used, the system must operate in a stateless manner and process each request independently without storing user data.

9.4 System Design Decisions

EduGenie follows a client-server architecture. A lightweight Flask framework was chosen to implement RESTful API endpoints efficiently. The decision to avoid a database was made to keep the application simple and focused on AI-driven processing rather than data storage. File-processing libraries such as PyPDF2, python-docx, python-pptx, and Pillow were selected to support commonly used educational document formats. The backend is responsible for input validation, text extraction, prompt construction, API communication, and response formatting, while the frontend handles user interaction and display. Waitress was selected as the production server instead of Flask's development server to ensure stable deployment.

9.5 Technology Stack Selection Justification

Python was selected due to its extensive ecosystem for AI integration and file processing. Flask was chosen because of its simplicity, flexibility, and suitability for REST API development. The Gemini Generative AI service from Google was selected to provide advanced text generation, summarization, and quiz creation capabilities. The google-generativeai library simplifies communication with the AI service. PyPDF2, python-docx, and python-pptx were chosen to extract text from academic documents, while Pillow supports image handling. The python-dotenv library ensures secure configuration management of API keys. Waitress was selected as a production-ready WSGI server to enhance deployment reliability.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

The EduGenie system was developed using Python 3.10 and the Flask web framework. A virtual environment was created to manage dependencies separately from the system Python installation. The development was carried out using Visual Studio Code as the Integrated Development Environment (IDE). An active internet connection is required to access the Gemini Generative AI API provided by Google.

Setup Commands

Open Command Prompt or Terminal and execute:

```
python --version
```

```
python -m venv venv
```

```
venv\Scripts\activate (Windows)
```

10.2 Dependency Installation

After activating the virtual environment, the required dependencies were installed using pip.

The following libraries are used in the project:

- Flask
- google-generativeai
- python-dotenv
- PyPDF2
- python-docx
- python-pptx
- Pillow
- waitress

Installation Command: `pip install -r requirements.txt`

10.3 Project Structure Creation

The project follows the below directory structure:

EduGenie/

```
|
|— app.py
|— requirements.txt
|— .env
|— templates/
|   |— index.html
|— static/
|   |— (CSS / JS files)
|— venv/
```

10.4 Configuration Setup

The Gemini API key is stored securely inside a .env file.

Create a .env file in the project root:

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

This milestone focuses on the implementation of the core functionalities of the EduGenie system. The main features include AI-based question answering, quiz generation, and document summarization. These features are implemented using a Flask backend integrated with the Gemini Generative AI API provided by Google.

11.1 Feature 1 – AI-Based Question Answering

Description:

This feature allows users to enter academic questions through the web interface and receive AI-generated answers. The system constructs a prompt based on the user input and sends it to the Gemini API, which returns a context-aware response. This feature helps students clarify doubts instantly without manual searching.

Code Explanation:

When a user submits a question, the Flask backend receives the request through the /ask endpoint. The input is validated to ensure it is not empty. A prompt is then constructed and sent to the Gemini API using the google-generativeai library. The AI-generated response is processed and returned to the frontend in JSON format

Screenshots / Outputs

11.2 Feature 2 – Quiz Generation

Description:

This feature enables automatic generation of multiple-choice questions (MCQs) from a given topic or uploaded document content. It is designed to support self-assessment and practice for students.

Code Explanation:

The quiz generation logic is implemented through the /quiz endpoint. The backend accepts a topic or extracted text from uploaded files. A structured prompt is created requesting the AI to generate MCQs in JSON format. The response received from the Gemini API is parsed and validated before being sent back to the frontend. Each quiz includes questions, options, correct answers, and explanations.

Screenshots / Outputs:

11.3 Feature 3 – Document Summarization

Description:

This feature allows users to upload documents such as PDFs, DOCX files, PPTX files, or images and receive concise summaries. It helps users quickly understand lengthy study materials.

Code Explanation:

The summarization feature is implemented using the /summary endpoint. Uploaded files are processed using appropriate libraries such as PyPDF2, python-docx, python-pptx, or Pillow to extract text. The extracted content is sent to the Gemini API with a summarization prompt. The generated summary is returned to the user as a text response.

Screenshots / Outputs:

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

All system components were integrated using a client–server architecture. The frontend sends user requests to the Flask backend, which processes input, extracts text from uploaded files, and communicates with the Gemini Generative AI API provided by Google. The AI response is formatted and returned to the user interface.

12.2 Performance Optimization

Performance was improved by validating inputs before API calls, minimizing unnecessary data processing, structuring prompts efficiently, and deploying the application using Waitress for stable request handling. The system remains lightweight and stateless for better speed and efficiency.

12.3 Security Enhancements

The API key is stored securely in a .env file and loaded using python-dotenv. Input and file validations are implemented to prevent invalid requests. No user data is stored, reducing security risks.

12.4 Error Handling & Validation

Error handling is implemented using validation checks and try–except blocks. The system manages invalid inputs, unsupported file formats, API failures, and JSON parsing errors gracefully without crashing.

13. MILESTONE 5: TESTING & VALIDATION

Test Case ID Input		Expected Output	Status
TC01	Valid question in /ask	AI-generated answer displayed	Passed
TC02	Topic in /quiz	Structured MCQ JSON response	Passed
TC03	Upload PDF file in /summary	Generated summary text	Passed
TC04	Empty input	Validation error message	Passed
TC05	Unsupported file format	File type error message	Passed

13.2 Unit Testing

Unit testing was performed on individual modules such as input validation, file extraction (PDF, DOCX, PPTX), API communication, and JSON parsing. Each function was tested independently to ensure correct output for valid and invalid inputs.

13.3 Integration Testing

Integration testing was conducted to verify that frontend forms, backend routes, file processing modules, and the Gemini Generative AI API provided by Google work together seamlessly. All endpoints (/ask, /quiz, /summary) were tested for proper request–response flow.

13.4 User Acceptance Testing (UAT)

The system was tested by students to ensure ease of use, accuracy of responses, and proper display of outputs. Feedback confirmed that the application is user-friendly and provides meaningful academic assistance.

13.5 Performance Metrics

- Average response time depends on API latency (internet-based).
- System runs smoothly on 4GB RAM.
- No database overhead, ensuring lightweight performance.

- Stable execution using Waitress in production mode.

14. DEPLOYMENT

14.1 Deployment Architecture

EduGenie follows a simple web deployment architecture. The client (user browser) sends HTTP requests to the deployed Flask backend server. The backend processes the request, communicates with the Gemini Generative AI API provided by Google, and returns the AI-generated response to the user. The system operates as a stateless web application without a database.

14.2 Hosting Platform

The application can be deployed on cloud platforms such as:

- Render
- Railway
- AWS EC2
- Google Cloud
- Heroku

14.3 Deployment Steps

8. Install Python and project dependencies.
9. Set up the .env file with the Gemini API key.
10. Activate the virtual environment.
11. Install required packages using `pip install -r requirements.txt`.
12. Run the application using Waitress:
13. `waitress-serve --host=0.0.0.0 --port=5000 app:app`
14. Configure hosting platform settings (port, environment variables).

14.4 Production Considerations

- Secure storage of API keys using environment variables.
- Use of Waitress instead of Flask development server.

- Proper input validation and file-type restrictions.
- Stable internet connection for API communication.
- Scalability depends on hosting platform resources.

16. RESULTS

• System Output

The EduGenie system successfully generates AI-based answers, structured MCQ quizzes, and concise summaries from text and uploaded documents. All endpoints (/ask, /quiz, /summary) function correctly and return meaningful responses using the Gemini Generative AI API provided by Google.

• Performance Evaluation

The system performs efficiently on a standard system (4GB RAM). Response time mainly depends on internet speed and external API latency. Since no database is used, internal processing is lightweight and fast.

• Screenshots

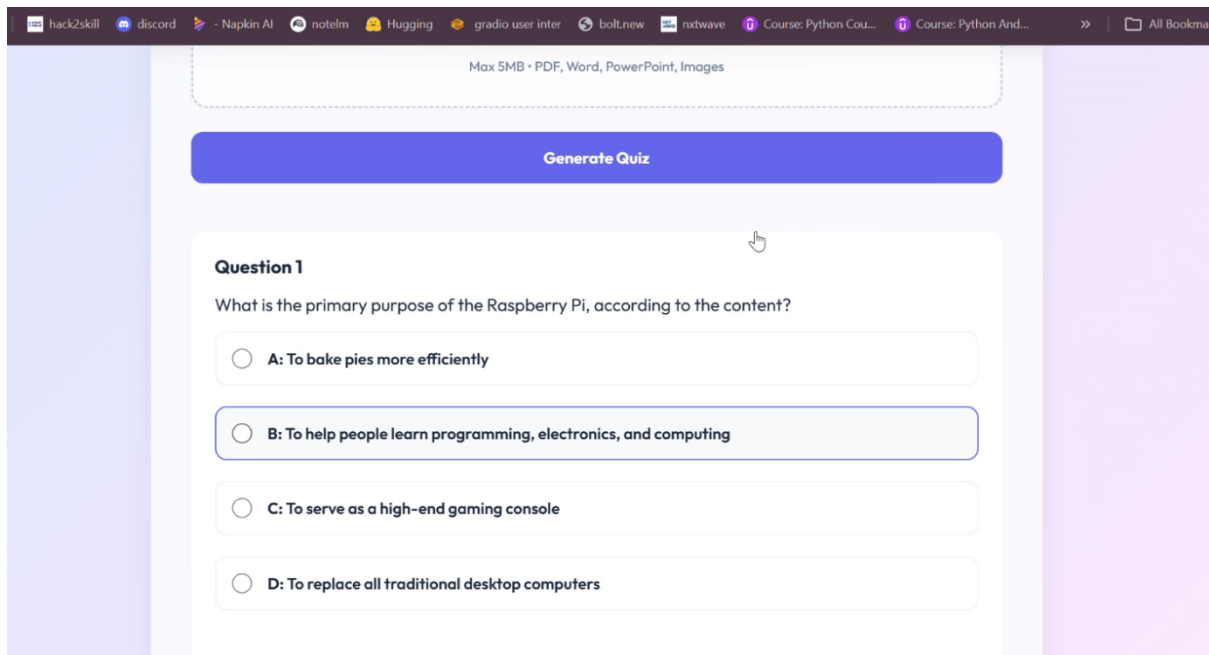


Fig2. Generated quiz

- **Benchmark Results**

- Average response time: Few seconds (API dependent)
- Low memory usage (stateless system)
- Stable performance during testing

17. ADVANTAGES & LIMITATIONS

Advantages

- Provides instant AI-based academic assistance.
- Supports question answering, quiz generation, and document summarization.
- Accepts multiple file formats (PDF, DOCX, PPTX, images).
- Lightweight and stateless (no database required).
- Secure API key management using environment variables.
- Easy deployment using Flask and Waitress.
- User-friendly web interface.

Limitations

- Requires stable internet connection (API dependent).
- Response time depends on external API latency.
- No user login or data storage functionality.
- Accuracy depends on AI-generated content.
- Limited scalability without advanced cloud infrastructure.

18. FUTURE ENHANCEMENTS

Scalability Improvements

The system can be enhanced by deploying it on scalable cloud infrastructure with load balancing and auto-scaling features. Adding a database for storing user history and quizzes would improve long-term usability.

Feature Expansion

Future versions can include user authentication, personalized learning dashboards, chat-based interaction, multilingual UI support, and advanced analytics for tracking learning progress.

Cloud Integration

EduGenie can be fully deployed on cloud platforms such as AWS or Google Cloud for better availability, scalability, and secure environment management. Containerization using Docker can also improve portability.

Mobile Integration

A dedicated Android or iOS application can be developed to provide mobile access. Alternatively, the web application can be converted into a Progressive Web App (PWA) for mobile-friendly usage.

Automation

Automation can be implemented for scheduled quiz generation, automatic content updates, usage analytics, and integration with Learning Management Systems (LMS) for seamless academic workflows.

19. CONCLUSION

● Summary of Implementation

EduGenie was successfully developed as a Flask-based web application integrating the Gemini Generative AI API provided by Google. The system implements AI-based question answering, quiz generation, and document summarization with support for multiple file formats. All core modules were integrated, tested, and deployed using a production-ready setup.

● Technical Achievements

The project demonstrates successful integration of a web framework with a generative AI service. It includes secure API key management, structured JSON output generation,

multimodal input handling, and deployment using Waitress. The system operates efficiently without a database, maintaining a lightweight and stateless architecture.

- Practical Impact

EduGenie provides instant academic assistance to students and educators, reducing manual effort in preparing notes and quizzes. It showcases the practical application of generative AI in education and serves as a scalable foundation for future EdTech solutions.